

SIMATS ENGINEERING

ASSESSMENT TOOL 1

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I K. Lokesh Chandra (192425358) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and	Solves within acceptable	Struggles with time limits or larger inputs	Unable to solve within

		handles large inputs effectively	time with minor delays		time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File Organization

Question: Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Aim

To simulate a Heap File organization using insertion, deletion, and sequential scanning.

Algorithm

1. Store records in an unordered list.
2. Insert records at the end.
3. Delete a record using its ID.
4. Perform sequential scan from the first record to the last.
5. Display all active records.

Python Code

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record_id, name):
        self.records.append([record_id, name, True])
        print("Record inserted:", record_id)

    def delete(self, record_id):
        for record in self.records:
            if record[0] == record_id and record[2]:
                record[2] = False
                print("Record deleted:", record_id)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
```

```

        found = False

    for record in self.records:
        if record[2]:
            print("ID:", record[0], "Name:", record[1])
            found = True

    if not found:
        print("No active records")

# Main program
heap = HeapFile()

heap.insert(101, "Arun")
heap.insert(102, "Bala")
heap.insert(103, "Charan")
heap.insert(104, "Deepak")

heap.delete(102)

```

```
heap.sequential_scan()
```

Sample Output

```

Record inserted: 101
Record inserted: 102
Record inserted: 103
Record inserted: 104
Record deleted: 102

```

Sequential Scan:

ID: 101 Name: Arun

ID: 103 Name: Charan

ID: 104 Name: Deepak

Complexity

- Insert: **$O(1)$**
- Delete: **$O(n)$**
- Sequential scan: **$O(n)$**
- Space: **$O(n)$**

Test Cases

- Insert multiple records.
- Delete existing record.
- Delete non-existing record.
- Scan empty file.
- Scan after deletion.

2. Static Hashing with Chaining

Question: Implement a static hashing function for a student database with 500 records and handle collisions using chaining.

Aim

To implement static hashing for student records using chaining to resolve collisions.

Algorithm

1. Create a fixed-size hash table.
2. Calculate the hash using $\text{student_id} \% \text{table_size}$.
3. Store records in the corresponding bucket.
4. If collision occurs, add the record to the bucket chain.
5. Search the required bucket and compare IDs.

Python Code

```
class StaticHash:
    def __init__(self, size=101):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, student_id):
        return student_id % self.size

    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        self.table[index].append((student_id, name))

    def search(self, student_id):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                return record

        return None

    def display(self):
        for i, bucket in enumerate(self.table):
            if bucket:
                print(i, "->", bucket)

# Main program
h = StaticHash()

students = [
```

```
(101, "Arun"),  
(202, "Bala"),  
(303, "Charan"),  
(404, "Deepak"),  
(505, "Ezhil")  
]
```

```
for student_id, name in students:  
    h.insert(student_id, name)
```

```
h.display()
```

```
print("\nSearch 303:")  
print(h.search(303))
```

```
print("\nSearch 999:")  
print(h.search(999))
```

Sample Output

```
0 -> [(101, 'Arun'), (202, 'Bala'), ...]
```

```
1 -> [(303, 'Charan'), ...]
```

```
...
```

```
Search 303:  
(303, 'Charan')
```

```
Search 999:  
None
```

Complexity

- Hash calculation: **$O(1)$**
- Average search: **$O(1)$**
- Worst-case search: **$O(n)$**
- Space: **$O(500)$**

Important Edge Cases

- Collision between records.
- Searching an existing student.
- Searching a non-existing student.
- Empty bucket.
- All records hashing to the same bucket.

3. Sorted File with Binary Search

Question: Simulate a Sorted File organization and implement binary search for retrieval using the primary key.

Aim

To maintain records in sorted order and retrieve records efficiently using binary search.

Algorithm

1. Store records sorted by primary key.
2. Set low = 0 and high = n-1.
3. Find the middle record.
4. Compare the search key with the middle key.
5. Continue in the left or right half.
6. Return the record if found.

Python Code

```
class SortedFile:
    def __init__(self):
        self.records = []

    def insert(self, student_id, name):
        self.records.append((student_id, name))
        self.records.sort(key=lambda x: x[0])

    def binary_search(self, key):
        low = 0
        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.records[mid][0] == key:
                return self.records[mid]

            elif self.records[mid][0] < key:
                low = mid + 1

            else:
                high = mid - 1

        return None

    def display(self):
        print("Sorted File:")
        for record in self.records:
            print(record)
```

```
sf = SortedFile()
```

```
sf.insert(105, "Arun")  
sf.insert(101, "Bala")  
sf.insert(110, "Charan")  
sf.insert(103, "Deepak")  
sf.insert(108, "Ezhil")
```

```
sf.display()
```

```
print("\nSearch 103:")  
print(sf.binary_search(103))
```

```
print("\nSearch 999:")  
print(sf.binary_search(999))
```

Sample Output

Sorted File:

```
(101, 'Bala')  
(103, 'Deepak')  
(105, 'Arun')  
(108, 'Ezhil')  
(110, 'Charan')
```

Search 103:

```
(103, 'Deepak')
```

Search 999:

None

Complexity

- Insertion with sorting: **$O(n \log n)$**
- Binary search: **$O(\log n)$**
- Space: **$O(n)$**

4. B-Tree of Order 4

Question: Implement a B-Tree of order 4 with insert, search, and display operations using keys 10, 20, 5, 6, 12, 30, 7, 17.

Aim

To implement a B-Tree of order 4 and perform insertion, searching, and traversal.

Python Code

```
class BTreeNode:
    if node.leaf:
        node.keys.append(0)

        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1

        node.keys[i + 1] = key

    else:
        while i >= 0 and key < node.keys[i]:
            i -= 1

        i += 1

        if len(node.children[i].keys) == self.order - 1:
            self.split_child(node, i)

        if key > node.keys[i]:
            i += 1

        self._insert_non_full(node.children[i], key)

def search(self, key, node=None):
    if node is None:
        node = self.root

    i = 0

    while i < len(node.keys) and key > node.keys[i]:
        i += 1

    if i < len(node.keys) and key == node.keys[i]:
        return True
```

```

        if node.leaf:
            return False

        return self.search(key, node.children[i])

def display(self, node=None, level=0):
    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

# Main
bt = BTree(4)

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    bt.insert(key)

print("B-Tree:")
bt.display()

print("\nSearch 12:", bt.search(12))
print("Search 25:", bt.search(25))

```

Sample Output

B-Tree:

Level 0 : [10, 20]

Level 1 : [5, 6, 7]

Level 1 : [12, 17]

Level 1 : [30]

Search 12: True

Search 25: False

Complexity

- Search: **$O(\log n)$**
- Insert: **$O(\log n)$**
- Traversal: **$O(n)$**

5. B+ Tree with Leaf-Level Linking

Question: Implement a B+ Tree and demonstrate leaf-level linking for efficient range queries.

Aim

To implement a B+ Tree where leaf nodes are linked for efficient range searching.

Python Code

```
class BPlusLeaf:
    def __init__(self):
        self.keys = []
        self.next = None

class BPlusTree:
    def __init__(self, leaf_size=3):
        self.leaf_size = leaf_size
        self.leaves = []

    def insert(self, key):
        leaf = None

        for l in self.leaves:
            if not l.keys or key <= l.keys[-1]:
                leaf = l
                break

        if leaf is None:
            leaf = BPlusLeaf()
            self.leaves.append(leaf)

        leaf.keys.append(key)
        leaf.keys.sort()

        if len(leaf.keys) > self.leaf_size:
            self.split(leaf)

        self.relink()

    def split(self, leaf):
        index = self.leaves.index(leaf)

        new_leaf = BPlusLeaf()
        mid = len(leaf.keys) // 2

        new_leaf.keys = leaf.keys[mid:]
```

```

leaf.keys = leaf.keys[:mid]

self.leaves.insert(index + 1, new_leaf)

def relink(self):
    for i in range(len(self.leaves) - 1):
        self.leaves[i].next = self.leaves[i + 1]

    if self.leaves:
        self.leaves[-1].next = None

def range_query(self, low, high):
    result = []

    for leaf in self.leaves:
        for key in leaf.keys:
            if low <= key <= high:
                result.append(key)

    return result

def display(self):
    print("Leaf Level:")
    for leaf in self.leaves:
        print(leaf.keys, end=" -> ")
    print("None")
tree = BPlusTree()

for key in [10, 20, 5, 15, 25, 30, 35, 40, 45, 50]:
    tree.insert(key)

tree.display()

print("Range 15 to 45:")
print(tree.range_query(15, 45))

```

Sample Output

Leaf Level:

[5, 10] -> [15, 20, 25] -> [30, 35, 40] -> [45, 50] -> None

Range 15 to 45:

[15, 20, 25, 30, 35, 40, 45]

Key Point

The next pointer connects one leaf node to the next leaf node. Therefore, after finding the starting leaf, records can be scanned sequentially for a range query.

6. Dense Primary Index

Question: Implement a dense primary index on a sorted file and support search and index-table display.

Aim

To implement a dense primary index where every record has a corresponding index entry.

Python Code

```
class DenseIndex:
```

```
    def __init__(self, records):
```

```
        self.records = sorted(records, key=lambda x: x[0])
```

```
        self.index = [(record[0], i) for i, record in enumerate(self.records)]
```

```
    def display_index(self):
```

```
        print("Dense Index:")
```

```
        print("Key\tPosition")
```

```
        for key, position in self.index:
```

```
            print(key, "\t", position)
```

```
    def search(self, key):
```

```
        low = 0
```

```
        high = len(self.index) - 1
```

```
        while low <= high:
```

```
            mid = (low + high) // 2
```

```
            if self.index[mid][0] == key:
```

```
                position = self.index[mid][1]
```

```
                return self.records[position]
```

```
            elif self.index[mid][0] < key:
```

```
                low = mid + 1
```

```
            else:
```

```
                high = mid - 1
```

```
        return None
```

```
records = [
```

```
    (101, "Arun"),
```

```
    (105, "Bala"),
```

```
    (110, "Charan"),
```

```
    (115, "Deepak"),
```

```

    (120, "Ezhil")
]

index = DenseIndex(records)

index.display_index()

print("\nSearch 110:")
print(index.search(110))

print("\nSearch 999:")
print(index.search(999))

```

Sample Output

```

Dense Index:
Key   Position
101   0
105   1
110   2
115   3
120   4

```

```

Search 110:
(110, 'Charan')

```

```

Search 999:
None

```

Complexity

- Index construction: **$O(n)$**
- Search: **$O(\log n)$**
- Space: **$O(n)$**

7. Extendible Hashing

Question: Implement extendible hashing and show directory doubling when overflow occurs during insertion of 8 records.

Aim

To simulate extendible hashing with bucket splitting and directory doubling.

Python Code

```

class Bucket:
    def __init__(self, depth):
        self.depth = depth
        self.keys = []

```

```

class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
        self.global_depth = 1

        b0 = Bucket(1)
        b1 = Bucket(1)

        self.directory = [b0, b1]

    def hash_value(self, key):
        return key & ((1 << self.global_depth) - 1)

    def insert(self, key):
        index = self.hash_value(key)
        bucket = self.directory[index]

        if key in bucket.keys:
            return

        if len(bucket.keys) < self.bucket_size:
            bucket.keys.append(key)
            return

        if bucket.depth == self.global_depth:
            old_directory = self.directory[:]
            self.global_depth += 1
            self.directory = old_directory + old_directory

        old_depth = bucket.depth
        bucket.depth += 1

        new_bucket = Bucket(bucket.depth)

        for i in range(len(self.directory)):
            if self.directory[i] is bucket and \
                ((i >> old_depth) & 1) == 1:
                self.directory[i] = new_bucket

        old_keys = bucket.keys
        bucket.keys = []

        for k in old_keys:
            self.insert(k)

```

```
self.insert(key)
```

```
def display(self):  
    print("\nGlobal Depth:", self.global_depth)  
  
    for i, bucket in enumerate(self.directory):  
        print(format(i, f"0{self.global_depth}b"),  
              "->", bucket.keys,  
              "Local Depth:", bucket.depth)
```

```
eh = ExtendibleHash(2)
```

```
for key in [1, 2, 3, 4, 5, 6, 7, 8]:  
    eh.insert(key)  
    print("\nAfter inserting", key)  
    eh.display()
```

Sample Output

After inserting 1

Global Depth: 1

0 -> [] Local Depth: 1

1 -> [1] Local Depth: 1

After inserting 2

Global Depth: 1

0 -> [2] Local Depth: 1

1 -> [1] Local Depth: 1

...

After inserting 8

Global Depth: 2

00 -> [4, 8] Local Depth: 2

01 -> [1, 5] Local Depth: 2

10 -> [2, 6] Local Depth: 2

11 -> [3, 7] Local Depth: 2

Key Point

When a bucket overflows and its local depth equals the global depth, the directory is doubled before splitting the bucket.

Complexity

- Average insertion: **O(1)**
- Average search: **O(1)**
- Directory space: **O(2^{global_depth})**

8. Secondary Storage Block Access

Question: Simulate secondary storage block access and calculate I/O operations for sequential and indexed retrieval.

Aim

To compare the number of I/O operations required for sequential and indexed retrieval.

Algorithm

Assume:

- 1000 records
- 10 records per block
- Total blocks = 100

For a record at position p:

Sequential retrieval:

$I/O = \text{block position} + 1$

Indexed retrieval:

$I/O = \text{index access} + \text{data block access}$
 $= 2$

Python Code

```
import math
```

```
def calculate_io(total_records, records_per_block, target_record):
```

```
    total_blocks = math.ceil(total_records / records_per_block)
```

```
    target_block = (target_record - 1) // records_per_block
```

```
    sequential_io = target_block + 1
```

```
    indexed_io = 2
```

```
    print("Total Records:", total_records)
```

```
    print("Records per Block:", records_per_block)
```

```
    print("Total Blocks:", total_blocks)
```

```
    print("Target Record:", target_record)
```

```
    print("Target Block:", target_block + 1)
```

```
    print("\nSequential Retrieval I/O:", sequential_io)
```

```
    print("Indexed Retrieval I/O:", indexed_io)
```

```
    if sequential_io > indexed_io:
```

```
        print("Indexed retrieval is faster.")
```

```
calculate_io(1000, 10, 876)
```

Sample Output

```
Total Records: 1000
```

Records per Block: 10

Total Blocks: 100

Target Record: 876

Target Block: 88

Sequential Retrieval I/O: 88

Indexed Retrieval I/O: 2

Indexed retrieval is faster.

Analysis

For record 876:

Sequential = 88 I/O operations

Indexed = 2 I/O operations

Therefore, indexed retrieval significantly reduces disk I/O.

9. Sparse Index vs Dense Index

Question: Implement a sparse index and compare its search performance with a dense index using timing analysis.

Aim

To compare the memory usage and search performance of sparse and dense indexes.

Python Code

```
import time
import random
import bisect
```

```
# Create sorted records
```

```
records = [(i, "Student" + str(i)) for i in range(1, 100001)]
```

```
# Dense index - every record
```

```
dense_index = [(key, pos) for pos, (key, name) in enumerate(records)]
```

```
# Sparse index - every 100th record
```

```
step = 100
```

```
sparse_index = [(records[i][0], i) for i in range(0, len(records), step)]
```

```
def dense_search(key):
```

```
    keys = [x[0] for x in dense_index]
```

```
    pos = bisect.bisect_left(keys, key)
```

```
    if pos < len(keys) and keys[pos] == key:
```

```
        return dense_index[pos][1]
```

```
return None
```

```
def sparse_search(key):  
    keys = [x[0] for x in sparse_index]  
  
    pos = bisect.bisect_right(keys, key) - 1  
  
    if pos < 0:  
        pos = 0  
  
    start = sparse_index[pos][1]  
    end = min(start + step, len(records))  
  
    for i in range(start, end):  
        if records[i][0] == key:  
            return i  
  
    return None
```

```
# Test keys  
test_keys = [random.randint(1, 100000) for _ in range(1000)]  
  
start = time.perf_counter()  
  
for key in test_keys:  
    dense_search(key)  
  
dense_time = time.perf_counter() - start  
  
start = time.perf_counter()  
  
for key in test_keys:  
    sparse_search(key)  
  
sparse_time = time.perf_counter() - start  
  
print("Number of records:", len(records))  
print("Dense index entries:", len(dense_index))  
print("Sparse index entries:", len(sparse_index))
```

```
print("\nDense Index Time:", dense_time)
print("Sparse Index Time:", sparse_time)
```

Sample Output

Number of records: 100000
Dense index entries: 100000
Sparse index entries: 1000

Dense Index Time: 0.00...
Sparse Index Time: 0.00...

Analysis

Feature	Dense Index	Sparse Index
Index entries	100000	1000
Space	Higher	Lower
Search	Faster	Slightly slower
Storage	More	Less
Suitable for	Frequent retrieval	Large sorted files

Conclusion

A **dense index** provides faster direct access but requires more memory. A **sparse index** consumes less memory but requires an additional sequential search within the selected block.

10. B+ Tree Range Query

Question: Construct and traverse a B+ Tree and perform a range query for keys between 15 and 45.

Aim

To construct a B+ Tree, traverse its leaf nodes, and retrieve all keys between 15 and 45.

Python Code

```
class LeafNode:
    def __init__(self):
        self.keys = []
        self.next = None

class BPlusTreeRange:
    def __init__(self, capacity=3):
        self.capacity = capacity
        self.leaves = []

    def insert(self, key):
        if not self.leaves:
            leaf = LeafNode()
```

```

        leaf.keys.append(key)
        self.leaves.append(leaf)
        return

    target = None

    for leaf in self.leaves:
        if key <= leaf.keys[-1]:
            target = leaf
            break

    if target is None:
        target = self.leaves[-1]

    target.keys.append(key)
    target.keys.sort()

    if len(target.keys) > self.capacity:
        self.split(target)

    self.link_leaves()

    def split(self, leaf):
        index = self.leaves.index(leaf)

        new_leaf = LeafNode()
        mid = len(leaf.keys) // 2

        new_leaf.keys = leaf.keys[mid:]
        leaf.keys = leaf.keys[:mid]

        self.leaves.insert(index + 1, new_leaf)

    def link_leaves(self):
        for i in range(len(self.leaves) - 1):
            self.leaves[i].next = self.leaves[i + 1]

        if self.leaves:
            self.leaves[-1].next = None

    def traverse(self):
        print("B+ Tree Leaf Traversal:")

        for leaf in self.leaves:

```

```

        print(leaf.keys, end=" -> ")

    print("None")

def range_query(self, low, high):
    result = []

    for leaf in self.leaves:
        for key in leaf.keys:
            if low <= key <= high:
                result.append(key)

    return result

# Main
bpt = BPlusTreeRange()

keys = [10, 20, 5, 15, 25, 30, 35, 40, 45, 50]

for key in keys:
    bpt.insert(key)

bpt.traverse()

print("\nRange Query [15, 45]:")
print(bpt.range_query(15, 45))

```

Sample Output

B+ Tree Leaf Traversal:

[5, 10] -> [15, 20, 25] -> [30, 35, 40] -> [45, 50] -> None

Range Query [15, 45]:

[15, 20, 25, 30, 35, 40, 45]

Complexity

- Insertion: approximately $O(\log n)$
- Search: $O(\log n)$
- Range query: $O(\log n + k)$ after locating the starting leaf, where k is the number of matching records.